🎯1️⃣ Nguyên tắc thiết kế (rất quan trọng)

❗ **Với Yjs:**

KHÔNG lưu từng edit
KHÔNG lưu operation

👉 **Chỉ lưu:**

metadata + snapshot (nếu cần)

🧩2️⃣ Collections chính

1. users

```
{
 _id: ObjectId,
 name: String,
 email: String,
 password: String,
 avatar: String,

 createdAt: Date,
 updatedAt: Date
}
```

Index

email: unique

---

2. documents ⭐ (quan trọng nhất)

```
{
 _id: ObjectId,

 title: String,
```

```
 owner: ObjectId, // userId

 // Yjs snapshot (optional - nâng cao)
 ydocState: Buffer,

 isDeleted: Boolean,

 createdAt: Date,
 updatedAt: Date
}
```

💡 **ydocState là gì?**

binary state của Yjs document

Dùng để:

- **load nhanh document**
- **backup**
- **restore**

👉 **KHÔNG bắt buộc (có thể thêm sau)**

---

3. document_permissions

```
{
 _id: ObjectId,

 documentId: ObjectId,
 userId: ObjectId,

 role: "owner" | "editor" | "viewer",

 createdAt: Date
}
```

Index (rất quan trọng)

{ documentId: 1, userId: 1 } unique

---

4. document_meta (optional nhưng xịn)

```
{
 _id: ObjectId,

 documentId: ObjectId,

 lastOpenedBy: ObjectId,
 lastOpenedAt: Date,

 activeUsers: Number
}
```

---

5. audit_logs (nếu muốn ăn điểm)

```
{
 _id: ObjectId,

 documentId: ObjectId,
 userId: ObjectId,

 action: "create" | "delete" | "share",

 createdAt: Date
}
```

🧠 3️⃣ Quan hệ giữa các collection

```
User
└── owns → Document

User
└── access → Document (via document_permissions)

Document
```

└── metadata → document_meta

⚡4️⃣ Flow dữ liệu thực tế

Khi mở document

GET /documents/:id
↓
check permission
↓
return metadata
↓
Yjs load state (WebSocket)

Khi edit

KHÔNG gọi API
↓
Yjs xử lý
↓
WebSocket sync

Khi save snapshot (optional)

server lấy Yjs state
↓
update document.ydocState

🚀5️⃣ Index quan trọng (bắt buộc)

documents

owner: 1
createdAt: -1

---

document_permissions

userId: 1
documentId: 1

💣6️⃣ Sai lầm nhiều nhóm mắc

❌ **Lưu:**

content mỗi lần gõ

❌ **Lưu:**

operation (insert, delete)

❌ **Dùng MongoDB để sync realtime**

👉 **ĐÚNG là:**

MongoDB → chỉ lưu state
WebSocket + Yjs → sync realtime

🎯7️⃣ Version nâng cao (nếu muốn top điểm)

Thêm collection:

document_versions

```
{
 _id: ObjectId,
 documentId: ObjectId,

 snapshot: Buffer, // Yjs state

 createdBy: ObjectId,
```

createdAt: Date
}

👉 **dùng cho:**

version history (giống Google Docs)

🧠 8️⃣ Tổng kết architecture DB

users
documents
document_permissions
document_meta (optional)
document_versions (optional)
audit_logs (optional)

💡 Lời khuyên leader

Bạn nên làm theo level:

MVP (đủ điểm cao)

users
documents
document_permissions

Advanced (ăn điểm)

+ ydocState
+ version
+ audit_logs

Nếu bạn muốn, mình có thể:

- **viết luôn Mongoose schema chuẩn MVC cho bạn**
- **hoặc vẽ ERD diagram đẹp để đưa vào báo cáo**

→ cái này giúp bạn ăn điểm phần thiết kế hệ thống rất mạnh.

1. Yêu cầu chức năng

- **Tạo và quản lý tài liệu**
  - người dùng có thể tạo, mở và chỉnh sửa tài liệu văn bản.
- **Chỉnh sửa cộng tác theo thời gian thực**
  - nhiều người dùng có thể chỉnh sửa cùng một tài liệu.
  - thay đổi của một người phải được **cập nhật tới các người dùng khác gần thời gian thực**.
- **Đồng bộ thay đổi**
  - hệ thống phải đảm bảo **tính nhất quán của tài liệu** khi nhiều người chỉnh sửa đồng thời.
- **Quản lý người dùng đang chỉnh sửa**
  - hiển thị danh sách người dùng đang tham gia chỉnh sửa.
  - hiển thị vị trí con trỏ hoặc vùng chỉnh sửa của từng người dùng (tùy chọn).
- **Lưu trữ tài liệu**
  - tài liệu phải được lưu trên server hoặc hệ thống lưu trữ phân tán.
- lịch sử chỉnh sửa và **versioning**
- **undo/redo** trong môi trường cộng tác
- hiển thị **con trỏ và vùng chọn của người dùng khác**
- cơ chế **offline editing và đồng bộ lại khi kết nối lại**
- phân quyền người dùng (owner, editor, viewer)

2. Yêu cầu kỹ thuật

Hệ thống phải thể hiện rõ các yếu tố của hệ thống phân tán:

- giao tiếp mạng giữa client và server (WebSocket)
- xử lý **nhiều client đồng thời**
- cơ chế **đồng bộ thay đổi giữa các client**
- xử lý các xung đột chỉnh sửa đồng thời

build TipTap editor integrated with Yjs collaboration using WebSocket provider

techstack: React, Vite, TipTap, Yjs, y-websocket, Zustand, React Router DOM, Axios, Tailwind CSS, …

Cấu trúc thư mục

```
src/
│
├── assets/              # ảnh, icon, svg
```

```
│
├── components/          # component dùng chung
│   ├── common/
│   │   ├── Button.jsx
│   │   ├── Input.jsx
│   │   └── Modal.jsx
│   │
│   ├── layout/
│   │   ├── Navbar.jsx
│   │   └── Sidebar.jsx
│   │
│   └── editor/
│       ├── EditorToolbar.jsx
│       ├── UserCursor.jsx
│       └── EditorWrapper.jsx
│
├── pages/               # page chính
│   ├── LoginPage.jsx
│   ├── DashboardPage.jsx
│   └── EditorPage.jsx
│
├── hooks/               # custom hooks
│   ├── useSocket.js
│   ├── useEditor.js
│   └── useAuth.js
│
├── services/            # call API / websocket
│   ├── api.js
│   ├── authService.js
│   ├── documentService.js
│   └── socketService.js
│
├── yjs/                 # phần quan trọng nhất (realtime)
│   ├── yjsConfig.js
│   ├── provider.js
│   └── awareness.js
│
```

```
├── store/              # state global
│   ├── authStore.js
│   └── documentStore.js
│
├── utils/
│   ├── constants.js
│   ├── helpers.js
│   └── debounce.js
│
├── routes/
│   └── AppRoutes.jsx
│
├── App.jsx
└── main.jsx
```

FEATURE LIST – COLLABORATIVE DOCUMENT EDITOR

1. Authentication

1.1 Login

- Người dùng nhập email + password
- Gửi request tới backend
- Nhận JWT token
- Lưu token (localStorage / cookie)

1.2 Register (optional)

- Tạo tài khoản mới

- Validate input (email, password)

2. Document Management

2.1 Create Document

- User tạo document mới
- Gán owner
- Khởi tạo nội dung rỗng

2.2 List Documents (Dashboard)

- Hiển thị danh sách document của user
- Có thể:
    - mở document
    - xóa document
    - đổi tên document

2.3 Open Document

- Load document theo ID
- Fetch metadata (title, owner)

- Khởi tạo editor

2.4 Delete Document

- Xóa document khỏi database
- Cập nhật UI ngay

🧠 3. Collaborative Editing (CORE)

3.1 Realtime Editing

- Nhiều user chỉnh sửa cùng lúc
- Nội dung update ngay lập tức

Flow:

User A edit
→ Yjs update
→ WebSocket gửi
→ server relay
→ User B nhận
→ merge → render

3.2 Conflict-free Synchronization

- Không mất dữ liệu khi nhiều user edit

- Dùng CRDT (Yjs)
- Merge tự động

3.3 Multiple User Cursor

- Hiển thị cursor của user khác
- Mỗi user có:
  - màu riêng
  - tên

3.4 User Presence (Online Users)

- Hiển thị danh sách user đang edit
- Update realtime khi user join/leave

⚡ 4. Offline Editing

4.1 Local Editing

- User vẫn edit khi mất mạng
- Nội dung lưu local (Yjs state)

4.2 Reconnect Sync

- Khi có mạng lại:
    - gửi update lên server
    - merge với dữ liệu hiện tại

4.3 Conflict Handling

- Không bị ghi đè dữ liệu
- Merge tự động bằng CRDT

🧾 5. Editor Features

5.1 Rich Text Editing

- Bold
- Italic
- Heading
- List (bullet, ordered)
- Link (optional)

5.2 Auto Save

- Không cần nút Save
- Sync liên tục qua WebSocket

5.3 Undo / Redo (Collaborative)

- Undo chỉ ảnh hưởng user hiện tại
- Không undo thay đổi của user khác

6. Authorization

6.1 Role-based Access

- Owner
- Editor
- Viewer

6.2 Share Document

- Mời user khác vào document
- Set quyền edit/view

🧩 7. UI / UX Features

7.1 Dashboard UI

- Danh sách document
- Button tạo document

7.2 Editor Layout

- Toolbar (format text)
- Editor area
- User list (online)

7.3 Loading State

- Loading khi fetch document
- Skeleton UI

⚙️ 8. System Behavior

8.1 WebSocket Connection

- Auto connect khi mở document

- Reconnect khi mất mạng

8.2 Room-based Editing

- Mỗi document là 1 room
- User join room theo documentId

8.3 Error Handling

- Mất mạng
- Token hết hạn
- Document không tồn tại

9. Minimum Feature Set

- Login
- Dashboard
- Create document
- Open document
- Realtime editing
- Multiple users
- Cursor hiển thị
- Auto sync
- Offline editing
- Undo/redo collaborative

- User roles
- Share document
- Version history (optional)

API CONTRACT – COLLABORATIVE DOCUMENT EDITOR

1. Authentication

POST /auth/register

Request

```
{
 "name": "string",
 "email": "string",
 "password": "string"
}
```

Response

```
{
 "message": "User created successfully",
 "user": {
   "id": "string",
   "name": "string",
   "email": "string"
 }
}
```

POST /auth/login

Request

```
{
 "email": "string",
```

```
 "password": "string"
}
```

Response

```
{
 "token": "jwt_token",
 "user": {
   "id": "string",
   "name": "string",
   "email": "string"
 }
}
```

GET /auth/me

Headers

Authorization: Bearer <token>

Response

```
{
 "id": "string",
 "name": "string",
 "email": "string"
}
```

2. Document Management

POST /documents

Headers

Authorization: Bearer <token>

Request

```
{
 "title": "Untitled Document"
}
```

Response

```
{
 "id": "doc_id",
 "title": "Untitled Document",
 "ownerId": "user_id",
 "createdAt": "timestamp"
}
```

GET /documents

Headers

Authorization: Bearer <token>

Response

```
[
 {
   "id": "doc_id",
   "title": "Document 1",
   "ownerId": "user_id",
   "updatedAt": "timestamp"
 }
]
```

GET /documents/:id

Headers

Authorization: Bearer <token>

Response

{
 "id": "doc_id",
 "title": "Document 1",
 "ownerId": "user_id",
 "createdAt": "timestamp",
 "updatedAt": "timestamp"
}

PATCH /documents/:id

Headers

Authorization: Bearer <token>

Request

{
 "title": "New Title"
}

Response

{
 "message": "Document updated"
}

DELETE /documents/:id

Headers

Authorization: Bearer <token>

Response

{
 "message": "Document deleted"
}

3. Realtime Collaboration (WebSocket)

Endpoint

ws://localhost:1234

Room Mapping

room = documentId

Client Connect

ws://localhost:1234/<documentId>

Data Flow

User edit
→ Yjs update
→ WebSocket send
→ Server relay
→ Other clients receive
→ Yjs merge → render

Important Rules

- Server does NOT handle merge logic
- Yjs handles all synchronization (CRDT)
- Each document = 1 WebSocket room

4. User Presence (Cursor / Online Users)

Data Structure

```
{
 "user": {
   "id": "user_id",
   "name": "John",
   "color": "#ff0000"
 },
 "cursor": {
   "from": 10,
   "to": 12
 }
}
```

Behavior

- User join → broadcast presence
- User leave → remove from list
- Cursor position sync realtime

5. Permission

POST /documents/:id/share

Request

```
{
 "userId": "user_id",
 "role": "editor"
}
```

GET /documents/:id/permissions

Response

```
[
 {
   "userId": "user_id",
   "role": "owner"
```

```
 },
 {
   "userId": "user_id_2",
   "role": "editor"
 }
]
```

6. Error Handling

Standard Error Format

```
{
 "error": "Error message"
}
```

Common Status Codes

401 Unauthorized
403 Forbidden
404 Not Found
500 Internal Server Error

---

7. System Rules

- Use REST API for authentication and document management
- Use WebSocket (y-websocket) for realtime collaboration
- DO NOT send document content via REST API
- Document content is managed entirely by Yjs
- Each document is mapped to a WebSocket room using documentId

Yjs Integration Rules:

- Create one Y.Doc per document
- Use WebsocketProvider(ws://localhost:1234, documentId, ydoc)
- Use TipTap Collaboration extension with ydoc
- Use CollaborationCursor for user presence

- Initialize Yjs ONLY inside EditorPage
- Destroy provider when leaving page

Page Flow Spec

Login Page

- User enters email/password
- Call POST /auth/login
- Store JWT token
- Redirect to /dashboard

Dashboard Page

- Call GET /documents
- Display list of documents
- Click document → navigate to /documents/:id
- Create document → POST /documents

Editor Page (QUAN TRỌNG NHẤT)

- Get documentId from URL
- Call GET /documents/:id (metadata only)
- Create Y.Doc
- Connect WebSocket provider with documentId
- Initialize TipTap editor with Yjs
- Render editor
- Cleanup on unmount

Editor Integration Rules:

- Use TipTap Collaboration extension with Y.Doc
- Disable default history extension
- Use CollaborationCursor with awareness from provider
- Create only ONE Y.Doc per document
- Destroy WebsocketProvider and Y.Doc on unmount

- Prevent multiple provider instances per document